



在上一篇博客里大概整理了tensorRT对于固定结构网络进行推理的流程，在申请内存的时候可以通过 `engine_` 拿到输入输出的结构，从而申请对应的内存，将GPU与CPU的内存指针分别保存在 `device_ptr` 与 `host_ptr` 中，推理的时候将输入tensor数据传送到GPU的指定位置上，还要指定大小。

```
const cudaError_t err = cudaMemcpyAsync(tensor.device_ptr, tensor.host_ptr, tensor.size, cudaMemcpyDeviceToHost, stream);
```

对于固定尺寸、固定批次的模型，这种方案是非常简单高效的，但是他的扩展性似乎不太行（我叫他扩展性，我不知道工业界叫他什么），就比如yolo模型部署，固定尺寸输入：1 * 3 * 640 * 640，一次只处理一张图像，如果想处理多路监控，还得多次部署，不光占用显存，还浪费了GPU的并行能力。所以还是希望他能做到动态的批次，让输入为：[?, 3, 640, 640]，根据具体的输入数据推断批次结构，申请内存，进行后续的推理，这个流程会涉及到两个问题：1、tensorRT如何支持动态批次？ 2、动态批次根据输入每次都要重新申请内存，是否会带来不必要的延迟？

注意，本文目前只讨论动态批次的处理方式，对于输入的一维float数组可以推断批次，设定输入形状；对于多维度动态，需要明确输入的形状，进行设定。

onnx支持动态维度？

是的，他支持的，动态维度的onnx一般会显示：[-1, 3, 640, 640]，其中批次维度就是动态的。对于导出的多动态维度模型，也能给它修改了，仅支持动态批次，以YOLO的导出与修改为例：

```
from ultralytics import YOLO
import onnx
from onnxruntime.tools.symbolic_shape_infer import SymbolicShapeInference

# 1. 先导出全动态模型
model = YOLO("yolov8m.pt")
model.export(format="onnx", dynamic=True, imgsz=640)

# 2. 加载 ONNX 模型
onnx_model = onnx.load("yolov8m.onnx")

# 3. 创建新的模型，修改输入输出
import onnx.helper as helper

# 获取原始图信息
graph = onnx_model.graph
initializers = list(graph.initializer)
nodes = list(graph.node)

# 创建新的输入（只有 batch 是动态的）
new_input = helper.make_tensor_value_info(
    'images',
    onnx.TensorProto.FLOAT,
    ['batch', 3, 640, 640] # 只有 batch 是符号维度
)

# 创建新的输出（只有 batch 是动态的）
new_output = helper.make_tensor_value_info(
    'output0',
    onnx.TensorProto.FLOAT,
    ['batch', 84, 8400] # 只有 batch 是符号维度
)

# 创建新图
new_graph = helper.make_graph(
    nodes,
    'yolov8m',
    [new_input],
    [new_output],
```

```
        initializers
    )

# 创建新模型
new_model = helper.make_model(new_graph)
new_model.opset_import.extend(onnx_model.opset_import)

# 保存
onnx.save(new_model, "yolov8m_batch_dynamic.onnx")
```

tensorRT的动态shape

tensorRT提供了一个用于设定动态尺寸的配置文件：`profile`，在明确启用动态维度，并且网络的输入tensor `nvInfer1::ITensor* input = network->getInput(i);` 存在某个维度为-1，在满足这个条件的情况下，我们就需要给网络设定 `profile`。举个例子：

假设当前输入tensor为动态维度: [-1, 3, 640, 640]，tensorRT部署是并不能让你随意输入第一个维度的大小，批次一定是存在一个合理的上下限。最小为1，最大受限于实际的业务需求、显存能力、tensorRT优化代价，可以设定一个上限为16，基于此，用三个确定的尺寸锚定一个区间，[1, 3, 640, 640] - [16, 3, 640, 640]，将这个固定的尺寸信息放到 `profile` 中，最大、最，再加一个最优，三个尺度。tensor构建引擎时会针对这个区间的输入进行优化，也只支持这个范围内的输入。

```

// 创建优化配置文件
nvinfer1::IOptimizationProfile* profile = builder->createOptimizationProfile();

const int num_inputs = network->getNbInputs();
for (int i = 0; i < num_inputs; ++i) {
    nvinfer1::ITensor* input = network->getInput(i);
    const std::string input_name = input->getName();
    const nvinfer1::Dims input_dims = input->getDimensions();

    // 检查是否有动态维度
    bool has_dynamic_dim = false;
    for (int j = 0; j < input_dims.nbDims; ++j) {
        if (input_dims.d[j] == -1) {
            has_dynamic_dim = true;
            break;
        }
    }

    if (has_dynamic_dim) {

        nvinfer1::Dims min_dims = input_dims;
        nvinfer1::Dims opt_dims = input_dims;
        nvinfer1::Dims max_dims = input_dims;

        min_dims.d[0] = 1;
        opt_dims.d[0] = config.max_batch_size / 2;
        max_dims.d[0] = config.max_batch_size;

        profile->setDimensions(input_name.c_str(), nvinfer1::OptProfileSelector::kMIN, min_dims);
        profile->setDimensions(input_name.c_str(), nvinfer1::OptProfileSelector::kOPT, opt_dims);
        profile->setDimensions(input_name.c_str(), nvinfer1::OptProfileSelector::kMAX, max_dims);
    }
}
builder_config->addOptimizationProfile(profile);

```

配置文件在设定区间端点，一共是三个形参，其中第一个是我们要设定的tensor的name，这个信息在onnx模型可视化软件中看到。

这里还存在一个问题，引擎推理时是如何实现动态批次支持的？这里其实在执行推理之前手动设

定输入tensor的形状，根据输入的数据长度、输入tensor的基础形状[-1, 3, 640, 640]，可以推断出当前输入数据是多个批次，将推测出的形状设定为计算结构：

```
nvinfer1::Dims inferred_dims = inferDimsFromData(tensor, input.second.size());
if (!context_->setInputShape(tensor.name.c_str(), inferred_dims)) {
    setLastError("Failed to set input shape for: " + tensor.name);
    return false;
}
```

动态批次的内存管理

固定批次时，我们可以预分配输入输出的buffer，上一篇文章中的 `allocateBuffers` 就是在构建引擎时就给分配好的内存区域（GPU&&CPU），输入数据先拷贝到这个buffer中，再传输到GPU上，输出同理。但现在输入数据批次为区间[1 - 16]，一个能想到的较为方便的解决方法：按照最优的尺寸申请内存，遇到超过最优的再去申请新的。

这像什么？C++容器中的两个概念：`size()` 与 `capacity()`，我们按照opt最优的去申请内存，得到的就是`capacity()`，而`size()`就是实际使用的容量，当此时来了一个`batch_size = 9`的，略大于opt申请的`capacity()`，此时就需要重新申请内存：

```

// 输入为一维数组，推断批次大小
nvinfer1::Dims inferred_dims = inferDimsFromData(tensor, input.second.size());
if (!context_>setInputShape(tensor.name.c_str(), inferred_dims)) {
    setLastError("Failed to set input shape for: " + tensor.name);
    return false;
}

// 更新当前使用的维度，但保持原始dims不变，原始的dim = [-1, 3, 640, 640]还要用来后续计算批次
tensor.current_dims = inferred_dims;

// 计算当前需要的size
const size_t required_size = getElementSize(tensor.data_type) * getDimsSize(inferred_dims);

// 检查是否需要重新分配内存
if (required_size > tensor.allocated_size) {
    // 重新分配更大的缓冲区
    if (!reallocateTensorBuffer(tensor, required_size)) {
        setLastError("Failed to reallocate buffer for tensor: " + tensor.name);
        return false;
    }
}

// 更新当前使用的size（但不超过allocated_size）
tensor.size = required_size;

```

对于输出的tensor也是，我们也用同样的方式去申请内存，不过比起输入tensor要推断结构，输出的tensor的形状不需要再次推断，在前面执行过 `context_>setInputShape(tensor.name.c_str(), inferred_dims)` 后，引擎是能推导出输出的形状的：

```

// 获取输出的实际维度（tensorRT会基于已设置的输入shape推导输出的shape）
nvinfer1::Dims output_dims = context_->getTensorShape(tensor.name.c_str());

const size_t required_size = getElementSize(tensor.data_type) * getDimsSize(output_dims);

// 检查是否需要重新分配
if (required_size > tensor.allocated_size) {
    if (!reallocateTensorBuffer(tensor, required_size)) {
        setLastError("Failed to reallocate buffer for output tensor: " + tensor.name);
        return false;
    }
}

// 更新当前维度和size
tensor.current_dims = output_dims;
tensor.size = required_size;

```

剩下就把他当固定结构的一样去做推理与复制结果即可。